Project Aegis

基于闭环控制理论的生产级 Agent 系统设计与实战

技术指导手册 | 适用于 200-1000 人规模科技企业

文档撰写者：盛少 (Ares Sheng)

Version 2.0 | 2026 年 4 月

分类：企业智能体架构 | 机密等级：内部公开

第 1 章 开篇：从“开环脚本”到“闭环控制系统”

在 Project Aegis 立项之初，我们对业界主流的开源 Agent 框架（如早期的 ReAct 范式实现）进行了深度的调研与压测。我们发现，这些框架在处理真实的线上 SRE（自动化运维）场景时，无一例外地走向了崩溃。

1.1 痛点直击：开环（Open-Loop）带来的灾难

当前开源生态最大的架构缺陷在于，它们本质上是一个开环脚本系统。模型输出一个文本指令，系统解析执行，把标准输出（stdout）塞回给模型，然后祈祷模型能明白发生了什么。在复杂的微服务故障排查中，这种缺乏对外部环境“真实状态反馈”的设计，会导致极其可怕的幻觉累积。

> ⚠ **踩坑警告：开环幻觉累积**
>
> 一旦环境状态发生微小偏移（例如命令执行超时、网络抖动），开环 Agent 就会疑狂重试或产生灾难性的长尾操作。我们在压测中观察到，某开源框架在网络抖动后的 15 分钟内连续发出了 47 次无效的 kubectl delete 命令。

1.2 架构破局：分布式闭环控制系统

为了根治这一痛点，我们决定在 Project Aegis 中彻底将 Agent 重新定义为一个类似于 Kubernetes Controller 的分布式闭环控制系统。大模型仅仅是系统中的一个非确定性计算节点。

◆ **图 1-1: Project Aegis 闭环控制架构全景**

```
graph TB
    subgraph Sensory["感知与状态面 Sensory & State Plane"]
        SS["State Streaming
K8s APIServer / Prometheus"]
        GDB["Graph DB (L2 Memory)
微服务拓扑树"]
        ES["Event Sourcing
不可变追加事件流"]
        SNAP["Snapshot Engine
时序记忆重构"]
    end

    subgraph Control["决策控制面 Control Plane"]
        ACTOR["Actor 状态机引擎
异步事件驱动"]
        ROUTE["Intent Router
```

```
感知 > 解析 > 规划 > 挂起 > 回调"]
        BFT["BFT 共识
多模型 Quorum 投票"]
    end

    subgraph Actuation["执行面 Actuation Plane"]
        GATE["Zero-Trust 执行网关
Intent Descriptor 解析"]
        SB["Ephemeral Sandbox
最小权限 Capability Token"]
        EBPF["eBPF 探针
Syscall 级拦截"]
    end

    subgraph Feedback["反馈与韧性 Feedback & Resilience"]
        CK["Checkpointing
分布式 K-V 持久化"]
        SAGA["SAGA 补偿
逆序回滚链"]
        DRIFT["Intent Drift Monitor
意图偏移监控"]
    end

    SS --> GDB --> ACTOR
    ES --> SNAP --> ACTOR
    ACTOR --> ROUTE --> BFT --> GATE
    GATE --> SB --> EBPF
    EBPF -->|"反馈回路"| Feedback
    Feedback -->|"状态对齐"| Sensory
```

↑ *粘贴到 mermaid.live 或任何 Markdown 渲染器查看*

> **源码审计参考：**我们同期审计了一个拥有近 5000 个源码文件的业界标杆 Agent 系统。其 42 个内建工具、14 步执行流水线、三层互不绕过的安全网证明了一个残酷事实：生产级 Agent 不是几百行 Python 能解决的问题。但它的单体循环架构在分布式场景下仍有局限，这正是 Aegis 要突破的方向。

第 2 章 感知与状态面：状态流捕获与时序记忆

传统的 Agent 开发往往纠结于“如何把上下文窗口塞满”或“如何做文本摘要压缩”。我们在真实的生产环境中发现，依赖大模型上下文来维持系统状态是一场架构灾难。

2.1 核心设计：上下文存储金字塔

我们将计算机体系结构中的存储金字塔引入了 Agent 设计。大模型的上下文窗口仅仅被视为“一级寄存器（L1 Register）”——它极速、珍贵但也极易溢出。

层级	类比	实现方案	容量	访问延迟
L1 寄存器	CPU Register	LLM 上下文窗口	128K-200K tokens	<100ms
L2 内存	RAM	Neo4j / JanusGraph 图数据库	百万级节点	5-50ms
L3 磁盘	SSD/HDD	Event Log (Kafka) + Checkpoint (Etcd)	无限追加	10-200ms

> **实操建议：选型建议**
> 200-500 人团队推荐 Neo4j Community Edition 作为 L2，它的 Cypher 查询语言对开发者友好。500-1000 人团队可考虑 JanusGraph + Cassandra 后端以获得更好的水平扩展性。L3 的 Event Log 用 Kafka 或 Pulsar，Checkpoint 用 Etcd 或 Redis Streams。

2.2 状态流捕获（State Streaming）

当 Aegis 接入线上集群时，它不是通过文本读取状态，而是通过监听 K8s APIServer 和 Prometheus 的状态流，在图数据库中实时维护一棵微服务拓扑树。大模型只在需要时，将图数据库中的局部子图“换页（Paging）”加载到上下文寄存器中。

TypeScript: 状态流捕获实现示例

```typescript
// 状态流捕获示例：监听 K8s Pod 状态变更，实时更新图数据库
async function watchPodState(kubeClient: KubeClient, graphDb: GraphDB) {
  const watcher = kubeClient.watch('/api/v1/pods');

  watcher.on('MODIFIED', (pod: Pod) => {
    // 将 Pod 状态变更写入图数据库
    graphDb.upsertNode({
      id: pod.metadata.uid,
      type: 'Pod',
      props: {
        name: pod.metadata.name,
        status: pod.status.phase,
```

```
      restartCount: pod.status.containerStatuses?.[0]?.restartCount,
      lastTransition: Date.now(),
    },
  });

  // 同时写入 Event Log（L3 层）
  eventLog.append({
    type: 'POD_STATE_CHANGE',
    timestamp: Date.now(),
    payload: { podId: pod.metadata.uid, newStatus: pod.status.phase },
  });
 });
}
```

2.3 踩坑经验：Event Sourcing 救了我们

> ⚠ **踩坑警告：生产事故复盘**
>
> 最初的 naive 实现导致了一个严重的生产事故：在排查一次复杂的跨可用区网络故障时，Aegis 经过 30 多轮深度检索，L1 寄存器被迫滚动清理。结果，Agent 竟然遗忘了最初触发它响应的 P0 级 OOM 报警上下文，开始在无关的日志堆里“梦游”。这次事故直接导致故障恢复时间从预期的 15 分钟延长到 2 小时。

为了根治这个问题，我们引入了事件溯源（Event Sourcing）机制。所有关键状态突变被序列化为不可变的追加事件流。当 L1 寄存器发生“缓存未命中”或严重偏离基线时，状态面通过回放 Event Log，瞬间重构出包含时间序列的记忆快照，强行对齐 Agent 的认知基线。

> **源码审计参考：**审计发现，业界标杆系统采用了 4 级退避压缩策略（Snip → Micro → Collapse → Auto Compact）来管理上下文溢出，并用 SYSTEM_PROMPT_DYNAMIC_BOUNDARY 把静态 Prompt 和动态 Prompt 物理分离以最大化 API Cache 命中率（节省级 73% 成本）。这是优秀的工程实践，但仍然是在 L1 层内做压缩。Aegis 的金字塔模型则从根本上重新定义了状态存储的分层架构。

第 3 章 决策控制面：Actor 模型与多路共识

3.1 核心设计：事件驱动的 Actor 引擎

在早期原型中，我们曾尝试用递归代码或死循环来驱动 Agent 思考。但这在分布式系统中极其脆弱。

> **源码审计参考：**审计发现，业界标杆系统的内核是一个 1729 行的 async generator，采用 while(true) + state 对象设计，包含 9 个状态转折点（包括熔断重试、模型降级、Token 预算 Nudge、Stop Hook 拦截等）。早期版本曾用递归调用自身，因长会话爆栈而重构。这证明了状态机比递归更具韧性——我们称之为“状态机内核的工程雏形”。

Project Aegis 在此基础上引入了基于 Actor 模型的事件驱动引擎。每个逻辑 Agent 是一个独立 Actor，拥有自己的信箱：

1. **感知（Perception）：**接收监控报警作为 Event 存入信箱
2. **意图解析（Routing）：**将 Event 转化为标准化的结构体
3. **规划生成（Planning）：**生成排查决策树
4. **挂起（Yield/Await）：**将高耗时动作下发后，Actor 立即挂起，释放计算资源
5. **回调（Callback）：**执行面完成动作后，通过事件总线唤醒 Actor 继续推进

> **实操建议：技术选型**
> 200-500 人团队推荐基于 Node.js 的 Actor 实现（如 xstate + BullMQ）。500-1000 人团队可考虑 Elixir/OTP 或 Akka (JVM)，它们提供原生的 Actor 监督树和故障迁移能力。

3.2 多路共识：BFT 隐喻下的多模型决断

在处理高危操作（如重启核心交易库、阻断特定 IP）时，我们不信任任何单一模型的决策。我们借用了拜占庭容错（BFT）思想：将高危意图组播给三个独立的专业化小模型，只有 2/3 节点投票通过才放行。

TypeScript: BFT 共识决断实现

```typescript
// BFT 共识决断伪代码
async function consensusGate(intent: IntentDescriptor): Promise<boolean> {
  // 将意图并行发送给 3 个独立的专业化模型
  const votes = await Promise.all([
    riskModel_DB.evaluate(intent),      // 数据库风险专家
    riskModel_Network.evaluate(intent),  // 网络风险专家
    riskModel_Blast.evaluate(intent),    // 爆炸半径评估
  ]);
```

```
  const approvals = votes.filter(v => v.decision === 'APPROVE').length;
  const quorum = Math.ceil(votes.length * 2 / 3); // 2/3 多数

  if (approvals >= quorum) {
    return true; // 放行
  }

  // 未达法定人数，升级到人工审批
  await escalateToHuman(intent, votes);
  return false;
}
```

第 4 章 执行面：零信任执行网关与 eBPF 沙箱

我们团队内部有一句极其严厉的架构军规：“大模型输出的任何字符串，都带有剧毒。”

4.1 核心设计：意图描述符与执行网关

大模型在 Aegis 中绝对不拥有实际的执行权限——它拿不到任何 API Key 或 Kubeconfig。大模型的输出仅仅被解析为一份被严格强类型的意图描述符（Intent Descriptor）。

TypeScript: 执行网关接口定义

```typescript
// 核心原则：非确定性模型只输出 Intent，确定性网关接管实际 Action
interface IntentDescriptor {
  readonly actionType: 'READ_METRICS' | 'MUTATE_STATE' | 'ESCALATE_TO_HUMAN';
  readonly targetResource: string;   // e.g., 'pod/payment-svc-xyz'
  readonly payload: unknown;
  readonly riskLevel: 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL';
  readonly consensusProof?: ConsensusSignature; // 高危操作需要 BFT 证明
}

interface IExecutionGateway {
  // 核心网关入口：将非确定性意图转化为受控沙箱中的物理执行
  dispatch(intent: IntentDescriptor): Promise<ExecutionResult>;

  // 物理隔离层：基于意图动态拉起 Ephemeral Sandbox
  provisionEphemeralSandbox(intent: IntentDescriptor): SandboxContext;

  // 内核级管控：通过 eBPF 挂载 Syscall 拦截器
  injectKernelProbes(sandboxId: string, rules: SyscallRule[]): void;
}
```

4.2 eBPF 沙箱的工作原理

执行网关收到 Intent 后，动态拉起一个短生命周期的沙箱容器，注入极小特权的 Capability Token。在 Node 层面部署 eBPF 探针：如果大模型产生幻觉，试图在沙箱中执行 rm -rf / 或向非白名单 IP 发起网络连接，eBPF 拦截器会在 Linux 内核层（Syscall 级别）直接阻断该行为。

> **源码审计参考：**审计中观察到，业界标杆系统的工具执行采用 14 步治理流水线（从 Zod Schema 校验到权限决策到遥测记录）、Fail-Closed 默认值设计（忘了声明并发安全性就默认串行）、三层互不绕过的安全网（Hook 的 allow 不能绕过 Settings 的 deny）。这是在应用层做治理的极致。Aegis 将治理下推到内核层（eBPF），实现物理隔离而非逻辑隔离。

4.3 流式执行与意图预解析

在分布式运维场景下，秒级的延迟可能导致故障扩散。我们做了一个关键的架构优化：意图预解析（Intent Pre-parsing）。网关在检测到模型输出的第一个合规指令片段时，就并行开始沙箱容器的冷启动与网络预拉取，而不是等待全文生成。这种“指令-环境并行加载”的设计将端到端响应延迟降低了 40%。

> **实操建议：实施路径**
> 小团队可以用 Docker + seccomp profile 作为第一步的沙箱方案，不需要立即上 eBPF。当系统负载超过 100 QPS 时，再升级到 eBPF。意图预解析可以用 JSON Schema 的流式解析器（如 jsonparse2 库）实现。

第 5 章 反馈层与韧性工程：异常崩溃后的自愈

系统设计最考验功力的不是正常运行，而是异常崩溃后的自愈。如果思考中途发生网络波动或 API 熔断，传统的“重头再来”会导致重复操作甚至状态污染。

5.1 检查点（Checkpointing）状态持久化

我们在控制面引入了检查点机制。Agent 的每一轮决策、每个生成的 Intent 以及执行面的每个反馈，都同步持久化到分布式 K-V 存储中。如果 Actor 节点发生故障迁移，新节点通过读取最新检查点瞬间恢复上下文和执行栈，实现“无损重试”。

5.2 SAGA 模式与事务补偿

大模型的指令往往涉及多个异构系统。如果 Aegis 执行了“扩容副本 → 修改权限 → 更新网关配置”这一系列操作，但在最后一步失败了，怎么办？

TypeScript: SAGA 事务补偿实现

```typescript
// SAGA 事务补偿示例
interface SagaStep {
  name: string;
  execute: () => Promise<StepResult>;
  compensate: () => Promise<void>;  // 每个 Action 必须配对 Compensation
}

async function executeSaga(steps: SagaStep[]): Promise<SagaResult> {
  const completed: SagaStep[] = [];

  for (const step of steps) {
    try {
      await step.execute();
      completed.push(step);
    } catch (error) {
      // 链条断裂：按逆序回滚所有已完成的步骤
      console.error(`Step "${step.name}" failed, rolling back...`);
      for (const done of completed.reverse()) {
        await done.compensate();
      }
      return { success: false, failedAt: step.name, error };
    }
```

```
  }
  return { success: true };
}

// 使用示例
const migrationSaga: SagaStep[] = [
  {
    name: 'scale-up-replicas',
    execute: () => kubectl.scale('deployment/api', { replicas: 5 }),
    compensate: () => kubectl.scale('deployment/api', { replicas: 3 }),
  },
  {
    name: 'update-rbac',
    execute: () => kubectl.apply(newRbacPolicy),
    compensate: () => kubectl.apply(originalRbacPolicy),
  },
  {
    name: 'update-ingress',
    execute: () => kubectl.apply(newIngressConfig),
    compensate: () => kubectl.apply(originalIngressConfig),
  },
];
```

源码审计参考：审计中观察到，业界标杆系统的 runAgent.ts（973 行）在每个子 Agent 结束时执行一套完整的清理链：Transcript 记录、元数据写入、Perfetto 追踪关闭、Shell 进程杀除、Hook 清理、文件状态清理、MCP 连接关闭。这是“Day 2 运维感知”的典型体现。Aegis 将清理链升级为 SAGA 补偿——不仅清理，还能回滚。

第 6 章 演进启示录：5 条生产级 Agent 架构原则

Project Aegis 从最初的 Demo 到支撑核心生产环境，结合对业界标杆系统的源码复盘，我们总结出五条足以写进研发军规的 Agent 架构原则。

原则 1：状态脱钩 (State Decoupling)

绝对不要把系统状态存储在大模型的上下文窗口中。模型是计算单元（CPU），不是存储单元（RAM）。状态必须外挂在可审计、可回滚的外部图数据库或事件流中。

原则 2：意图与执行的物理断裂

永远不要赋予 LLM 直接调用 API 的能力。架构上必须存在一个“零信任网关”，大模型只负责生成意图，物理操作必须由受控沙箱执行。

原则 3：用确定性架构围猎非确定性 AI

AI 的输出是非确定性的，但系统的防御链条、输入校验、权限判定、事务回滚必须是硬编码的、确定性的。这种“确定性外壳”是 Agent 系统进入工业级的唯一门票。

原则 4：观测意图而非观测文本

在做监控遥测时，盯着模型吐出的 Token 数量毫无意义。必须监控意图的偏移率（Intent Drift）和动作的成功率。我们要看 Agent “想做什么”与“实际做成了什么”之间的差距。

原则 5：生命周期清理链必须完整

Demo 跑通只是第一天。每个 Agent/子 Agent 在结束时必须执行完整的清理链：状态持久化、进程清理、连接释放、资源回收。没有清理链的 Agent 系统，第二天就会留下一堆僵尸进程和脏状态。

结语

Project Aegis 证明了 Agent 系统不仅仅是 Prompt Engineering 的产物，它更是一场严谨的分布式系统与闭环控制理论的胜利。我们不需要一个“更聪明”的模型，我们需要一个“更健壮”的系统框架来约束模型。

This document is part of the Project Aegis open-source architecture series.